

Introduction to Programming: Python – Course 3

Mengzhuo Guo

Academic Year 2026–2027

Contents

1	Learning Objectives	3
2	Program Flow	4
2.1	What Is Program Flow?	4
2.2	Three Basic Control Structures	4
3	Program Flowcharts	5
3.1	What Is a Flowchart?	5
3.2	Common Flowchart Symbols	5
3.3	A Sequential Flowchart	5
3.4	A Selection Flowchart	6
4	Sequential Structure	8
4.1	Definition	8
4.2	Example: Calculating a Total Price	8
4.3	Example: Temperature Conversion	8
5	Why Programs Need Selection Structures	8
6	Boolean Values and Boolean Expressions	9
6.1	The Boolean Data Type	9
6.2	Boolean Expressions	9
6.3	Storing a Boolean Result	10
6.4	Boolean Expressions in Everyday Language	10
7	Comparison Operators	10
7.1	Equality: ==	10
7.2	Assignment and Equality Are Different	11
7.3	Not Equal To: !=	11
7.4	Comparing Numbers	11
7.5	Comparing Strings	11
7.6	Chained Comparisons	12
8	Logical Operators	12
8.1	The and Operator	12
8.2	The or Operator	13
8.3	The not Operator	13
8.4	Combining Logical Operators	14
8.5	Operator Precedence	14
8.6	A Common Error with or	14

9	The Basic if Statement	15
9.1	General Syntax	15
9.2	Example: Positive Number	15
9.3	Understanding Indentation	16
9.4	The Colon	16
9.5	Multiple Statements in an if Block	17
9.6	Tracing an if Statement	17
10	The if-else Statement	17
10.1	General Syntax	17
10.2	Example: Pass or Fail	17
10.3	Example: Even or Odd	18
10.4	Example: Larger of Two Numbers	18
10.5	Assigning a Result in Each Branch	18
11	The if-elif-else Statement	18
11.1	Why Do We Need elif?	18
11.2	General Syntax	19
11.3	Example: Letter Grade	19
11.4	Why the Order Matters	20
11.5	Range Interpretation	20
11.6	Example: Comparing Two Numbers	21
11.7	Example: Temperature Description	21
11.8	Is else Required?	21
12	Independent if Statements versus an elif Chain	22
12.1	Independent if Statements	22
12.2	An if-elif-else Chain	22
12.3	When to Use Each Structure	22
13	Nested Conditional Statements	23
13.1	Definition	23
13.2	Example: Age and Ticket Check	23
13.3	Equivalent Condition Using and	23
13.4	When Nested Conditions Are Useful	24
13.5	Example: Positive, Negative, Even, or Odd	24
14	Input, Data Types, and Conditions	24
14.1	input() Returns a String	24
14.2	Incorrect String-Numeric Comparison	25
14.3	Standardizing Text Input	25
14.4	Checking Several Accepted Answers	25
15	Truthiness in Python	26
15.1	Example with a String	26
15.2	Explicit Beginner-Friendly Version	26
16	Common Conditional Patterns	26
16.1	Checking Whether a Number Is Inside a Range	26
16.2	Checking Whether a Number Is Outside a Range	27
16.3	Checking Divisibility	27
16.4	Checking Membership	27
16.5	Checking Several Requirements	27
16.6	Selecting One Category	27
16.7	Applying a Discount	28
16.8	Using a Boolean Flag	28
16.9	Finding the Maximum of Three Values	28

17 Boundary Conditions	28
17.1 Open and Closed Intervals	29
17.2 Testing Boundary Values	29
18 Detailed Worked Examples	29
18.1 Example 1: Determine the Sign of a Number	29
18.2 Example 2: Check Whether a Year Is a Leap Year	30
18.3 Example 3: Body Mass Index Category	30
18.4 Example 4: Shipping Fee	31
18.5 Example 5: Simple Login System	31
18.6 Example 6: Triangle Validity	32
18.7 Example 7: Triangle Type	32
18.8 Example 8: Electricity Bill	32
18.9 Example 9: Course Result with Validation	33
18.10Example 10: Rock, Paper, Scissors	33
19 Common Errors and How to Correct Them	34
19.1 Error 1: Using = Instead of ==	34
19.2 Error 2: Missing the Colon	34
19.3 Error 3: Incorrect Indentation	34
19.4 Error 4: Inconsistent Indentation	34
19.5 Error 5: Comparing Different Data Types	34
19.6 Error 6: Incorrect Use of <code>or</code>	35
19.7 Error 7: Overlapping Conditions in the Wrong Order	35
19.8 Error 8: Missing a Boundary	35
19.9 Error 9: Unnecessary Comparison with <code>True</code>	36
19.10Error 10: Confusing Separate <code>if</code> Statements with <code>elif</code>	36
19.11Error 11: Writing an Impossible Condition	36
19.12Error 12: Writing an Always-True Condition	36
20 How to Design an <code>if</code> Statement	37
20.1 Step 1: Identify the Input	37
20.2 Step 2: Identify the Possible Outcomes	37
20.3 Step 3: Identify the Decision Rule	37
20.4 Step 4: Choose the Correct Structure	37
20.5 Step 5: Check the Boundaries	37
20.6 Step 6: Trace the Program	38

1 Learning Objectives

Learning Objectives

After completing this chapter, students should be able to:

1. explain how a computer program executes statements;
2. distinguish sequential, selection, and repetition structures;
3. draw and understand basic program flowcharts;
4. understand the Boolean values `True` and `False`;
5. write comparison expressions using operators such as `==`, `!=`, `<`, `<=`, `>`, and `>=`;
6. combine conditions using `and`, `or`, and `not`;
7. write programs using `if`, `if-else`, and `if-elif-else`;

8. understand nested conditional statements;
9. identify boundary cases and choose correct conditions;
10. trace the execution of a conditional program;
11. find and correct common errors in conditional statements;
12. solve basic decision-making problems using Python.

This chapter focuses on one of the most important abilities of a computer program: **making decisions**.

A computer does not make decisions in the same way that a human does. Instead, it evaluates a condition. The result of the condition is either **True** or **False**. The program then chooses which statements to execute.

For example:

- If the temperature is below 0°C, display a freezing warning.
- If a student's score is at least 60, display "Pass".
- If a user enters the correct password, allow access.
- If a number is even, divide it by 2.
- If a customer spends more than 500 yuan, apply a discount.

These decisions are implemented using Python conditional statements, especially **if**, **elif**, and **else**.

2 Program Flow

2.1 What Is Program Flow?

Program flow describes the order in which the statements in a program are executed.

Consider the following program:

```
1 print("Step 1")
2 print("Step 2")
3 print("Step 3")
```

Python executes the program from top to bottom:

1. execute `print("Step 1");`
2. execute `print("Step 2");`
3. execute `print("Step 3").`

The output is:

```
1 Step 1
2 Step 2
3 Step 3
```

In a more complex program, the order may change because of decisions or loops.

2.2 Three Basic Control Structures

Most programs are constructed from three fundamental control structures.

1. Sequential structure

Statements are executed one after another, from top to bottom.

2. Selection structure

The program chooses one or more paths according to a condition.

3. Repetition structure

The program repeats a group of statements while a condition is true or for a specified number of times.

This chapter concentrates on the first two structures, especially the selection structure.

Structured Programming

A large program can usually be understood as a combination of sequence, selection, and repetition. Learning these three structures is therefore a fundamental step in programming.

3 Program Flowcharts

3.1 What Is a Flowchart?

A **flowchart** is a graphical representation of an algorithm or program process. It uses standard symbols to show:

- where the program begins and ends;
- what input is received;
- what processing is performed;
- what decisions are made;
- what output is produced;
- the order in which the steps are executed.

Flowcharts are useful because they help us understand the logic of a program before writing Python code.

3.2 Common Flowchart Symbols

Symbol	Name	Purpose
Oval or rounded rectangle	Start/End	Shows where an algorithm or program begins and ends.
Rectangle	Process	Represents a calculation, assignment, or processing step.
Parallelogram	Input/Output	Represents data entered by the user or information displayed by the program.
Diamond	Decision	Represents a condition with different possible outcomes, usually True/False or Yes/No .
Arrow	Flow line	Shows the direction and order of execution.

3.3 A Sequential Flowchart

The following flowchart describes a program that reads the length and width of a rectangle and calculates its area.

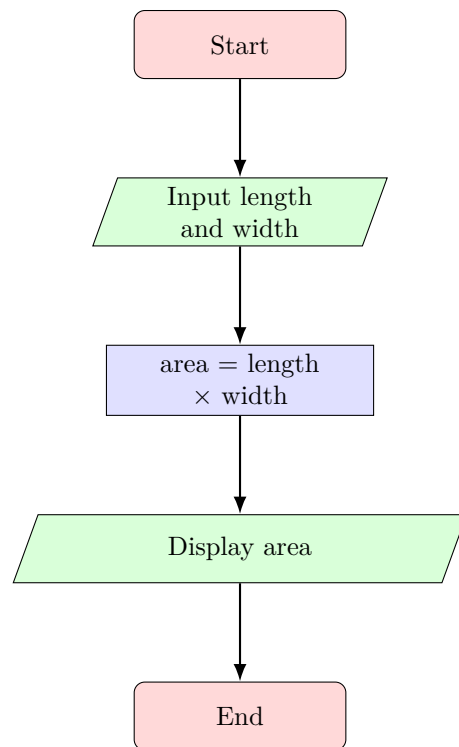


Figure 1: Flowchart of a sequential program

The corresponding Python program is:

```
1 length = float(input("Enter the length: "))
2 width = float(input("Enter the width: "))
3
4 area = length * width
5
6 print("The area is", area)
```

3.4 A Selection Flowchart

The following flowchart determines whether a student passes an examination.

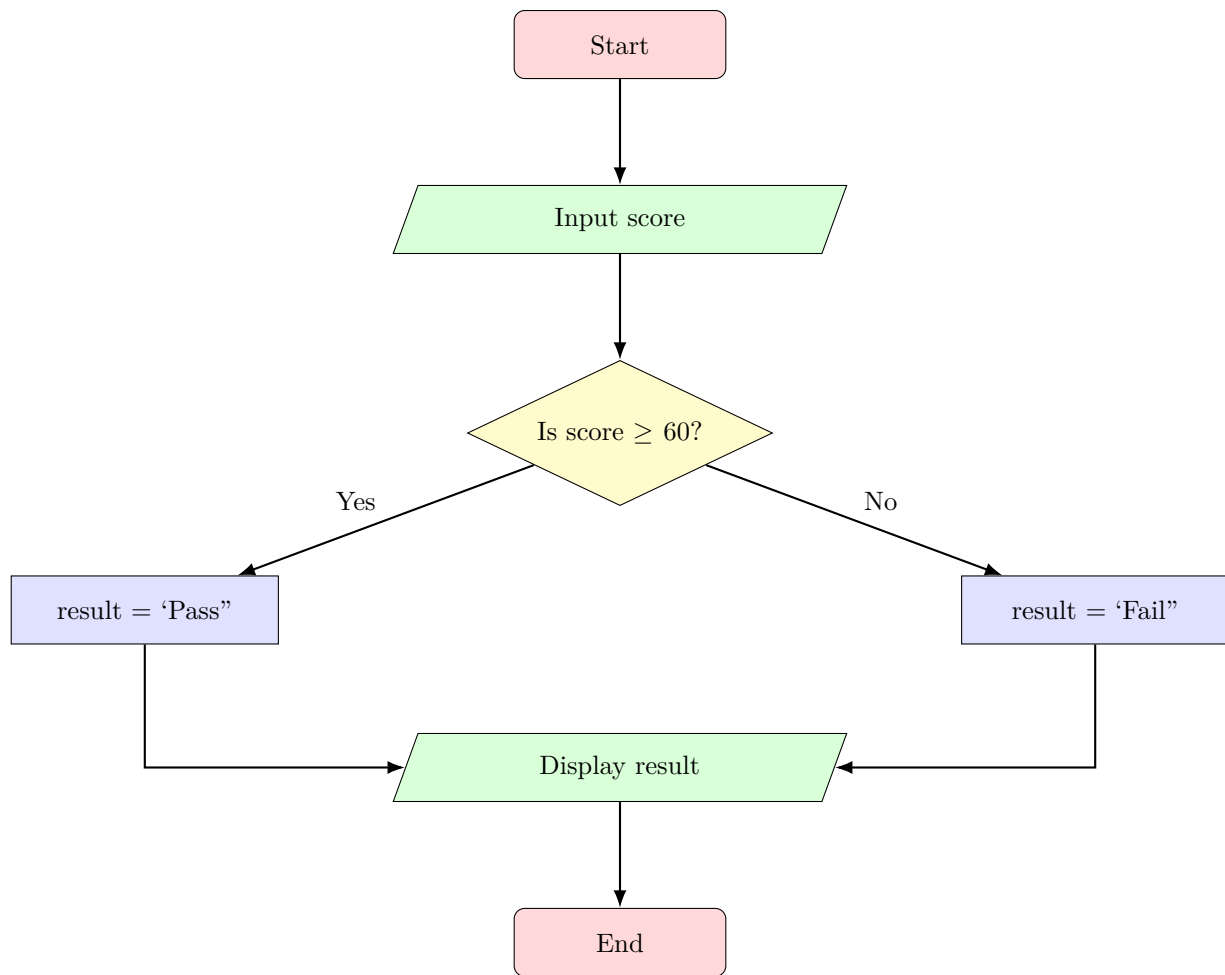


Figure 2: Flowchart of a selection structure

The corresponding Python program is:

```
1 score = float(input("Enter your score: "))
2
3 if score >= 60:
4     result = "Pass"
5 else:
6     result = "Fail"
7
8 print(result)
```

Flowchart Decision

A decision symbol normally has at least two outgoing paths. For a simple condition, these paths usually represent:

- Yes and No;
- True and False;
- condition satisfied and condition not satisfied.

4 Sequential Structure

4.1 Definition

In a sequential structure, statements are executed in the order in which they appear.

```
1 statement_1
2 statement_2
3 statement_3
```

Python normally starts with the first statement and moves downward one line at a time.

4.2 Example: Calculating a Total Price

Suppose a customer buys several identical products. The program receives the unit price and quantity and then calculates the total price.

```
1 price = float(input("Enter the unit price: "))
2 quantity = int(input("Enter the quantity: "))
3
4 total = price * quantity
5
6 print("Total price:", total)
```

The program performs four steps:

1. read the unit price;
2. read the quantity;
3. calculate the total;
4. display the result.

Every statement is executed exactly once.

4.3 Example: Temperature Conversion

The formula for converting Celsius temperature to Fahrenheit temperature is

$$F = \frac{9}{5}C + 32.$$

```
1 celsius = float(input("Enter the temperature in Celsius: "))
2
3 fahrenheit = 9 / 5 * celsius + 32
4
5 print("Temperature in Fahrenheit:", fahrenheit)
```

This program does not make a decision. Therefore, it uses only a sequential structure.

5 Why Programs Need Selection Structures

Sequential programs always execute the same statements. However, many real-world problems require different actions in different situations.

For example:

- A bank allows a withdrawal only when the account has enough money.
- A university gives different grades for different score ranges.
- A website displays an error message when a password is incorrect.
- A store offers a discount only when the purchase amount is large enough.

- A game gives a reward when a player reaches a target score.

To solve such problems, a program must evaluate a condition and select an appropriate path. The general logic is:

$$\text{Evaluate a condition} \longrightarrow \begin{cases} \text{perform one action,} & \text{if the condition is true,} \\ \text{perform another action,} & \text{if the condition is false.} \end{cases}$$

6 Boolean Values and Boolean Expressions

6.1 The Boolean Data Type

Python has a data type called `bool`. A Boolean value has only two possible values:

- `True`
- `False`

The first letter must be uppercase.

```
1 is_student = True
2 has_ticket = False
3
4 print(is_student)
5 print(has_ticket)
6 print(type(is_student))
```

Output:

```
1 True
2 False
3 <class 'bool'>
```

Capitalization Matters

The Boolean values must be written as `True` and `False`. The following are incorrect:

```
1 true
2 false
3 TRUE
4 FALSE
```

Python is case-sensitive.

6.2 Boolean Expressions

A **Boolean expression** is an expression whose result is either `True` or `False`.

Examples:

```
1 print(5 > 3)
2 print(10 == 10)
3 print(7 != 7)
4 print(4 <= 9)
```

Output:

```
1 True
2 True
3 False
4 True
```

Boolean expressions are the basis of conditional statements.

6.3 Storing a Boolean Result

A Boolean expression can be assigned to a variable.

```
1 age = 20
2 is_adult = age >= 18
3
4 print(is_adult)
```

The expression `age >= 18` is evaluated first. Because 20 is greater than or equal to 18, the result is `True`. Therefore, `is_adult` stores `True`.

6.4 Boolean Expressions in Everyday Language

Consider the statement:

The temperature is below zero.

In Python, this can be represented as:

```
1 temperature < 0
```

Similarly:

Everyday statement	Python Boolean expression
The score is at least 60.	<code>score >= 60</code>
The age is less than 18.	<code>age < 18</code>
The password is correct.	<code>password == correct_password</code>
The number is not zero.	<code>number != 0</code>
The balance is enough.	<code>balance >= withdrawal</code>
The answer is “yes”.	<code>answer == "yes"</code>

7 Comparison Operators

Comparison operators compare two values and produce a Boolean result.

Operator	Meaning	Example
<code>==</code>	Equal to	<code>5 == 5</code> produces <code>True</code> .
<code>!=</code>	Not equal to	<code>5 != 3</code> produces <code>True</code> .
<code>></code>	Greater than	<code>8 > 4</code> produces <code>True</code> .
<code><</code>	Less than	<code>2 < 9</code> produces <code>True</code> .
<code>>=</code>	Greater than or equal to	<code>60 >= 60</code> produces <code>True</code> .
<code><=</code>	Less than or equal to	<code>7 <= 10</code> produces <code>True</code> .

7.1 Equality: `==`

The operator `==` checks whether two values are equal.

```
1 x = 10
2
3 print(x == 10)
4 print(x == 8)
```

Output:

```
1 True
2 False
```

7.2 Assignment and Equality Are Different

Python uses:

- = for assignment;
- == for comparison.

```
1 x = 10      # Assign 10 to x
2 x == 10     # Check whether x is equal to 10
```

Do Not Confuse = and ==

The following code is incorrect:

```
1 if score = 100:
2     print("Perfect")
```

The correct code is:

```
1 if score == 100:
2     print("Perfect")
```

7.3 Not Equal To: !=

The operator != checks whether two values are different.

```
1 password = "python123"
2
3 print(password != "admin")
4 print(password != "python123")
```

Output:

```
1 True
2 False
```

7.4 Comparing Numbers

```
1 number = 15
2
3 print(number > 10)
4 print(number < 20)
5 print(number >= 15)
6 print(number <= 14)
```

Output:

```
1 True
2 True
3 True
4 False
```

7.5 Comparing Strings

Strings can also be compared.

```
1 answer = "yes"
2
3 print(answer == "yes")
4 print(answer == "Yes")
```

Output:

```
1 True
2 False
```

String comparisons are case-sensitive. Therefore, "yes" and "Yes" are different strings. A common solution is to standardize the user's input:

```
1 answer = input("Continue? ").strip().lower()
2
3 print(answer == "yes")
```

Here:

- `strip()` removes unnecessary spaces at the beginning and end;
- `lower()` converts letters to lowercase.

7.6 Chained Comparisons

Python allows chained comparisons.

```
1 age = 20
2
3 print(18 <= age <= 60)
```

This is equivalent to:

```
1 print(age >= 18 and age <= 60)
```

Another example:

```
1 score = 85
2
3 if 80 <= score < 90:
4     print("The score is between 80 and 89.")
```

Using Chained Comparisons

The expression

```
1 0 <= score <= 100
```

is clear and natural in Python. It checks whether `score` is inside the closed interval $[0,100]$.

8 Logical Operators

A program often needs to combine several Boolean expressions. Python provides three main logical operators:

- `and`
- `or`
- `not`

8.1 The and Operator

The expression

```
1 condition_1 and condition_2
```

is true only when both conditions are true.

Condition A	Condition B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Example:

```

1 age = 22
2 has_ticket = True
3
4 if age >= 18 and has_ticket:
5     print("You may enter.")

```

Both conditions must be true:

1. the person must be at least 18;
2. the person must have a ticket.

8.2 The or Operator

The expression

```

1 condition_1 or condition_2

```

is true when at least one condition is true.

Condition A	Condition B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Example:

```

1 day = "Saturday"
2
3 if day == "Saturday" or day == "Sunday":
4     print("It is the weekend.")

```

Only one of the two comparisons needs to be true.

8.3 The not Operator

The `not` operator reverses a Boolean value.

Condition A	not A
True	False
False	True

Example:

```

1 is_raining = False
2
3 if not is_raining:
4     print("You do not need an umbrella.")

```

Because `is_raining` is `False`, `not is_raining` is `True`.

8.4 Combining Logical Operators

```
1 age = 25
2 is_member = True
3 has_coupon = False
4
5 if age >= 18 and (is_member or has_coupon):
6     print("Discount available.")
```

The condition is true when:

1. the customer is at least 18; and
2. the customer is either a member or has a coupon.

Parentheses improve readability and make the intended logic clear.

8.5 Operator Precedence

The main precedence order is:

1. arithmetic operations;
2. comparison operations;
3. not;
4. and;
5. or.

For example:

```
1 x = 5
2
3 result = x > 0 and x < 10
```

Python first evaluates:

```
1 x > 0
2 x < 10
```

and then combines the two Boolean results using `and`.

Use Parentheses for Clarity

Even when parentheses are not strictly required, they can make a complex condition much easier to understand.

For example:

```
1 if (age >= 18) and (is_member or has_coupon):
2     print("Eligible")
```

8.6 A Common Error with or

The following code does not work as intended:

```
1 choice = 2
2
3 if choice == 1 or 2:
4     print("Valid choice")
```

The correct form is:

```
1 if choice == 1 or choice == 2:
2     print("Valid choice")
```

An even clearer form is:

```
1 if choice in (1, 2):
2     print("Valid choice")
```

Repeat the Complete Comparison

In a compound condition, each side should normally contain a complete Boolean expression.

Incorrect:

```
1 if month == 1 or 2 or 3:
2     print("First quarter")
```

Correct:

```
1 if month == 1 or month == 2 or month == 3:
2     print("First quarter")
```

Better:

```
1 if month in (1, 2, 3):
2     print("First quarter")
```

9 The Basic if Statement

9.1 General Syntax

The simplest conditional statement has the following structure:

```
1 if condition:
2     statement
```

A more general version is:

```
1 if condition:
2     statement_1
3     statement_2
4     statement_3
```

The indented statements form the **body** or **block** of the **if** statement.

The body is executed only when the condition is **True**.

9.2 Example: Positive Number

```
1 number = float(input("Enter a number: "))
2
3 if number > 0:
4     print("The number is positive.")
5
6 print("Program finished.")
```

Suppose the user enters 8.

1. Python evaluates `number > 0`.
2. The result is **True**.
3. The indented statement is executed.

4. The final statement is executed.

Output:

```
1 The number is positive.
2 Program finished.
```

Suppose the user enters -3.

1. Python evaluates `number > 0`.
2. The result is `False`.
3. The indented statement is skipped.
4. The final statement is still executed.

Output:

```
1 Program finished.
```

9.3 Understanding Indentation

Python uses indentation to identify the statements that belong to an `if` block.

```
1 temperature = -5
2
3 if temperature < 0:
4     print("Warning: freezing temperature.")
5     print("Please wear warm clothes.")
6
7 print("Weather check completed.")
```

The first two `print` statements are inside the `if` block. The final statement is outside the block.

Indentation Is Part of Python Syntax

The following program is incorrect:

```
1 if temperature < 0:
2 print("Freezing")
```

Python expects an indented block after the colon.

The correct version is:

```
1 if temperature < 0:
2     print("Freezing")
```

Four spaces are normally used for each indentation level.

9.4 The Colon

A colon must appear at the end of the `if` line.

Incorrect:

```
1 if score >= 60
2     print("Pass")
```

Correct:

```
1 if score >= 60:
2     print("Pass")
```

9.5 Multiple Statements in an if Block

```
1 balance = 1000
2 withdrawal = 300
3
4 if balance >= withdrawal:
5     balance = balance - withdrawal
6     print("Withdrawal successful.")
7     print("Remaining balance:", balance)
```

All three indented statements execute when the condition is true.

9.6 Tracing an if Statement

Consider:

```
1 x = 8
2
3 if x > 5:
4     x = x + 2
5     print(x)
6
7 print("End")
```

Execution trace:

Step	Statement	Result
1	x = 8	x becomes 8.
2	x > 5	The condition is True.
3	x = x + 2	x becomes 10.
4	print(x)	Displays 10.
5	print("End")	Displays End.

10 The if-else Statement

10.1 General Syntax

An if-else statement chooses between two paths.

```
1 if condition:
2     statements_when_true
3 else:
4     statements_when_false
```

Exactly one of the two blocks is executed.

10.2 Example: Pass or Fail

```
1 score = float(input("Enter your score: "))
2
3 if score >= 60:
4     print("Pass")
5 else:
6     print("Fail")
```

If the score is 75, the condition is true and the program displays **Pass**.

If the score is 45, the condition is false and the program displays **Fail**.

10.3 Example: Even or Odd

An integer is even when its remainder after division by 2 is zero.

```
1 number = int(input("Enter an integer: "))
2
3 if number % 2 == 0:
4     print("The number is even.")
5 else:
6     print("The number is odd.")
```

Examples:

$$8 \bmod 2 = 0, \quad 13 \bmod 2 = 1.$$

Therefore, 8 is even and 13 is odd.

10.4 Example: Larger of Two Numbers

```
1 a = float(input("Enter the first number: "))
2 b = float(input("Enter the second number: "))
3
4 if a > b:
5     print(a, "is larger.")
6 else:
7     print(b, "is larger or equal.")
```

This version does not distinguish between $a < b$ and $a == b$. A later section will show how to use `elif` to represent three different possibilities.

10.5 Assigning a Result in Each Branch

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     category = "adult"
5 else:
6     category = "minor"
7
8 print("Category:", category)
```

This is often clearer than placing a separate `print` statement in each branch.

11 The if-elif-else Statement

11.1 Why Do We Need elif?

Some problems have more than two possible outcomes.

For example, a student's result might be:

- Excellent;
- Good;
- Pass;
- Fail.

Python uses `elif`, meaning “else if”, to test additional conditions.

11.2 General Syntax

```
1 if condition_1:
2     statements_1
3 elif condition_2:
4     statements_2
5 elif condition_3:
6     statements_3
7 else:
8     statements_4
```

Python evaluates the conditions from top to bottom.

1. If `condition_1` is true, execute the first block and skip the remaining branches.
2. Otherwise, evaluate `condition_2`.
3. If `condition_2` is true, execute the second block and skip the remaining branches.
4. Continue in the same way.
5. If no previous condition is true, execute the `else` block.

Only one branch is executed.

11.3 Example: Letter Grade

```
1 score = float(input("Enter your score: "))
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 elif score >= 60:
10    grade = "D"
11 else:
12    grade = "F"
13
14 print("Your grade is", grade)
```

11.4 Why the Order Matters

Suppose the score is 95.

Python tests:

1. Is `score >= 90`? Yes.
2. Assign "A" to `grade`.
3. Skip all remaining branches.

Now consider the following incorrect order:

```
1 if score >= 60:
2     grade = "D"
3 elif score >= 70:
4     grade = "C"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 90:
8     grade = "A"
9 else:
10    grade = "F"
```

For a score of 95, the first condition `score >= 60` is already true. Therefore, the program assigns grade D and never reaches the later conditions.

Test More Specific or Higher Thresholds First

For descending score categories, start with the highest threshold.

Correct order:

```
1 if score >= 90:
2     ...
3 elif score >= 80:
4     ...
5 elif score >= 70:
6     ...
```

11.5 Range Interpretation

The grade program can be interpreted as follows:

Score range	Grade	Why
$90 \leq \text{score}$	A	The first condition is true.
$80 \leq \text{score} < 90$	B	The first condition is false, but the second is true.
$70 \leq \text{score} < 80$	C	The first two conditions are false.
$60 \leq \text{score} < 70$	D	The first three conditions are false.
$\text{score} < 60$	F	No previous condition is true.

Although the program only writes `score >= 80` in the second branch, Python reaches that branch only after determining that `score >= 90` is false. Therefore, the effective range is

$$80 \leq \text{score} < 90.$$

11.6 Example: Comparing Two Numbers

```
1 a = float(input("Enter the first number: "))
2 b = float(input("Enter the second number: "))
3
4 if a > b:
5     print("The first number is larger.")
6 elif a < b:
7     print("The second number is larger.")
8 else:
9     print("The two numbers are equal.")
```

There are three mutually exclusive possibilities:

$$a > b, \quad a < b, \quad a = b.$$

11.7 Example: Temperature Description

```
1 temperature = float(input("Enter the temperature: "))
2
3 if temperature < 0:
4     description = "Freezing"
5 elif temperature < 10:
6     description = "Cold"
7 elif temperature < 20:
8     description = "Cool"
9 elif temperature < 30:
10    description = "Warm"
11 else:
12    description = "Hot"
13
14 print(description)
```

Here the thresholds are arranged in ascending order because each condition uses `<`.

The effective ranges are:

- `temperature < 0`: Freezing;
- `0 <= temperature < 10`: Cold;
- `10 <= temperature < 20`: Cool;
- `20 <= temperature < 30`: Warm;
- `temperature >= 30`: Hot.

11.8 Is else Required?

No. An `if-elif` structure can be used without `else`.

```
1 command = input("Enter a command: ")
2
3 if command == "start":
4     print("Starting...")
5 elif command == "stop":
6     print("Stopping...")
```

If the command is neither `"start"` nor `"stop"`, the program does nothing.

However, an `else` branch is often useful for handling unexpected input.

```

1 command = input("Enter a command: ")
2
3 if command == "start":
4     print("Starting...")
5 elif command == "stop":
6     print("Stopping...")
7 else:
8     print("Unknown command.")

```

12 Independent if Statements versus an elif Chain

This distinction is extremely important.

12.1 Independent if Statements

Consider:

```

1 score = 95
2
3 if score >= 60:
4     print("Pass")
5
6 if score >= 80:
7     print("Good")
8
9 if score >= 90:
10    print("Excellent")

```

Output:

```

1 Pass
2 Good
3 Excellent

```

All three conditions are evaluated independently. More than one block can be executed.

12.2 An if-elif-else Chain

```

1 score = 95
2
3 if score >= 90:
4     print("Excellent")
5 elif score >= 80:
6     print("Good")
7 elif score >= 60:
8     print("Pass")
9 else:
10    print("Fail")

```

Output:

```

1 Excellent

```

After Python finds the first true condition, it skips the remaining branches.

12.3 When to Use Each Structure

Use separate if statements when several actions may all apply.

Example:

```

1 number = 12
2
3 if number > 0:
4     print("Positive")
5
6 if number % 2 == 0:
7     print("Even")
8
9 if number > 10:
10    print("Greater than 10")

```

All three descriptions are true.

Use an if-elif-else chain when exactly one category should be selected.

Example:

```

1 if number < 0:
2     print("Negative")
3 elif number == 0:
4     print("Zero")
5 else:
6     print("Positive")

```

Key Distinction

- Separate if statements: zero, one, or several blocks may execute.
- One if-elif-else chain: at most one block executes.

13 Nested Conditional Statements

13.1 Definition

A conditional statement placed inside another conditional statement is called a **nested conditional statement**.

General form:

```

1 if outer_condition:
2     if inner_condition:
3         statements

```

13.2 Example: Age and Ticket Check

```

1 age = int(input("Enter your age: "))
2 has_ticket = input("Do you have a ticket? ").strip().lower()
3
4 if age >= 18:
5     if has_ticket == "yes":
6         print("You may enter.")
7     else:
8         print("You need a ticket.")
9 else:
10    print("You are too young to enter.")

```

The inner condition is evaluated only when `age >= 18` is true.

13.3 Equivalent Condition Using and

The following nested structure:

```

1 if age >= 18:
2     if has_ticket == "yes":
3         print("You may enter.")

```

can often be simplified as:

```

1 if age >= 18 and has_ticket == "yes":
2     print("You may enter.")

```

The simpler version is usually preferred when the conditions can be naturally combined.

13.4 When Nested Conditions Are Useful

Nested conditions are useful when the second decision depends on the result of the first decision.

For example:

```

1 username = input("Username: ")
2 password = input("Password: ")
3
4 if username == "student":
5     if password == "python123":
6         print("Login successful.")
7     else:
8         print("Incorrect password.")
9 else:
10    print("Unknown username.")

```

The password is checked differently from the username because the program may need to provide different messages.

13.5 Example: Positive, Negative, Even, or Odd

```

1 number = int(input("Enter an integer: "))
2
3 if number > 0:
4     print("The number is positive.")
5
6     if number % 2 == 0:
7         print("The number is even.")
8     else:
9         print("The number is odd.")
10
11 elif number < 0:
12     print("The number is negative.")
13
14     if number % 2 == 0:
15         print("The number is even.")
16     else:
17         print("The number is odd.")
18
19 else:
20     print("The number is zero.")

```

This program demonstrates multiple indentation levels.

14 Input, Data Types, and Conditions

14.1 input() Returns a String

The `input()` function always returns a string.

```

1 age = input("Enter your age: ")
2
3 print(age)
4 print(type(age))

```

Even if the user enters 20, the value is stored as the string "20".
To perform a numerical comparison, convert the input:

```

1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("Adult")

```

14.2 Incorrect String-Numeric Comparison

The following code produces an error:

```

1 age = input("Enter your age: ")
2
3 if age >= 18:
4     print("Adult")

```

Here, `age` is a string, while 18 is an integer.
The correct version is:

```

1 age = int(input("Enter your age: "))

```

14.3 Standardizing Text Input

User input may contain uppercase letters or extra spaces.

```

1 answer = input("Continue? ").strip().lower()
2
3 if answer == "yes":
4     print("Continuing...")
5 else:
6     print("Stopping...")

```

The following inputs are all converted to "yes":

- yes
- YES
- Yes
- yes

14.4 Checking Several Accepted Answers

```

1 answer = input("Continue? ").strip().lower()
2
3 if answer in ("yes", "y"):
4     print("Continuing...")
5 elif answer in ("no", "n"):
6     print("Stopping...")
7 else:
8     print("Invalid answer.")

```

The operator `in` checks whether a value belongs to a collection.

15 Truthiness in Python

Python can treat some non-Boolean values as true or false in a condition.

Common false-like values include:

- False;
- 0;
- 0.0;
- the empty string "";
- empty containers such as [], (), and ;
- None.

Most other values are treated as true.

15.1 Example with a String

```
1 name = input("Enter your name: ")
2
3 if name:
4     print("Hello,", name)
5 else:
6     print("You did not enter a name.")
```

If name is an empty string, the condition is false.

15.2 Explicit Beginner-Friendly Version

For beginners, the following version may be easier to understand:

```
1 if name != "":
2     print("Hello,", name)
3 else:
4     print("You did not enter a name.")
```

Both versions are correct.

Recommended Learning Order

First learn to write explicit Boolean expressions such as:

```
1 name != ""
2 number != 0
3 items_count > 0
```

After the logic is clear, learn the shorter truthiness form.

16 Common Conditional Patterns

16.1 Checking Whether a Number Is Inside a Range

```
1 score = float(input("Enter a score: "))
2
3 if 0 <= score <= 100:
4     print("Valid score")
5 else:
6     print("Invalid score")
```

16.2 Checking Whether a Number Is Outside a Range

```
1 score = float(input("Enter a score: "))
2
3 if score < 0 or score > 100:
4     print("Invalid score")
5 else:
6     print("Valid score")
```

These two conditions are logical opposites:

$$0 \leq \text{score} \leq 100$$

and

$$\text{score} < 0 \quad \text{or} \quad \text{score} > 100.$$

16.3 Checking Divisibility

A number (a) is divisible by (b) when the remainder is zero.

```
1 number = int(input("Enter an integer: "))
2
3 if number % 3 == 0:
4     print("The number is divisible by 3.")
5 else:
6     print("The number is not divisible by 3.")
```

16.4 Checking Membership

```
1 day = input("Enter a day: ").strip().lower()
2
3 if day in ("saturday", "sunday"):
4     print("Weekend")
5 else:
6     print("Weekday")
```

16.5 Checking Several Requirements

```
1 age = int(input("Enter your age: "))
2 height = float(input("Enter your height in cm: "))
3
4 if age >= 12 and height >= 140:
5     print("You may ride.")
6 else:
7     print("You may not ride.")
```

16.6 Selecting One Category

```
1 income = float(input("Enter annual income: "))
2
3 if income < 30000:
4     category = "Low"
5 elif income < 100000:
6     category = "Medium"
7 else:
8     category = "High"
9
10 print("Income category:", category)
```

16.7 Applying a Discount

```
1 amount = float(input("Enter the purchase amount: "))
2
3 if amount >= 500:
4     discount_rate = 0.10
5 else:
6     discount_rate = 0.00
7
8 discount = amount * discount_rate
9 final_amount = amount - discount
10
11 print("Original amount:", amount)
12 print("Discount:", discount)
13 print("Final amount:", final_amount)
```

16.8 Using a Boolean Flag

A Boolean variable can record whether a condition is satisfied.

```
1 score = float(input("Enter your score: "))
2
3 passed = score >= 60
4
5 if passed:
6     print("Congratulations. You passed.")
7 else:
8     print("You did not pass.")
```

16.9 Finding the Maximum of Three Values

```
1 a = float(input("Enter a: "))
2 b = float(input("Enter b: "))
3 c = float(input("Enter c: "))
4
5 maximum = a
6
7 if b > maximum:
8     maximum = b
9
10 if c > maximum:
11     maximum = c
12
13 print("Maximum:", maximum)
```

Notice that this example uses two independent `if` statements.

17 Boundary Conditions

Many mistakes in conditional statements occur at boundaries.

Consider a rule:

Students with a score of 60 or above pass.

The correct condition is:

```
1 score >= 60
```

The condition

```
1 score > 60
```

is incorrect because a student with exactly 60 would fail.

17.1 Open and Closed Intervals

The condition

```
1 0 < x < 10
```

represents the open interval

[(0,10).]

The values 0 and 10 are excluded.

The condition

```
1 0 <= x <= 10
```

represents the closed interval

[[0,10].]

The values 0 and 10 are included.

17.2 Testing Boundary Values

For the condition

```
1 score >= 60
```

important test values include:

- 59: immediately below the boundary;
- 60: exactly on the boundary;
- 61: immediately above the boundary.

For the valid range

```
1 0 <= score <= 100
```

important test values include:

[-1, 0, 1, 99, 100, 101.]

Boundary Testing

When testing a conditional program, always include values:

- below the boundary;
- exactly on the boundary;
- above the boundary.

18 Detailed Worked Examples

18.1 Example 1: Determine the Sign of a Number

Problem: Read a number and determine whether it is positive, negative, or zero.

```

1 number = float(input("Enter a number: "))
2
3 if number > 0:
4     print("Positive")
5 elif number < 0:
6     print("Negative")
7 else:
8     print("Zero")

```

The three possibilities are mutually exclusive and collectively exhaustive:

$$number > 0, \quad number < 0, \quad number = 0.$$

18.2 Example 2: Check Whether a Year Is a Leap Year

A year is a leap year when:

- it is divisible by 400; or
- it is divisible by 4 but not divisible by 100.

```

1 year = int(input("Enter a year: "))
2
3 if year % 400 == 0:
4     print("Leap year")
5 elif year % 100 == 0:
6     print("Not a leap year")
7 elif year % 4 == 0:
8     print("Leap year")
9 else:
10    print("Not a leap year")

```

A compact equivalent condition is:

```

1 if year % 400 == 0 or (year % 4 == 0 and year % 100 != 0):
2     print("Leap year")
3 else:
4     print("Not a leap year")

```

Examples:

- 2000 is divisible by 400, so it is a leap year.
- 1900 is divisible by 100 but not by 400, so it is not a leap year.
- 2024 is divisible by 4 but not by 100, so it is a leap year.
- 2025 is not divisible by 4, so it is not a leap year.

18.3 Example 3: Body Mass Index Category

The body mass index is calculated as

$$BMI = \frac{\text{weight in kg}}{(\text{height in m})^2}.$$

For this programming exercise, use the following simplified categories:

- below 18.5: Underweight;
- 18.5 to below 24: Normal;
- 24 to below 28: Overweight;

- 28 or above: Obese.

```
1 weight = float(input("Enter weight in kg: "))
2 height = float(input("Enter height in meters: "))
3
4 bmi = weight / height ** 2
5
6 if bmi < 18.5:
7     category = "Underweight"
8 elif bmi < 24:
9     category = "Normal"
10 elif bmi < 28:
11     category = "Overweight"
12 else:
13     category = "Obese"
14
15 print("BMI:", round(bmi, 2))
16 print("Category:", category)
```

This is a programming example rather than medical advice.

18.4 Example 4: Shipping Fee

Suppose a store uses the following rule:

- purchase below 100 yuan: shipping fee is 10 yuan;
- purchase from 100 to below 300 yuan: shipping fee is 5 yuan;
- purchase of at least 300 yuan: free shipping.

```
1 amount = float(input("Enter the purchase amount: "))
2
3 if amount < 100:
4     shipping_fee = 10
5 elif amount < 300:
6     shipping_fee = 5
7 else:
8     shipping_fee = 0
9
10 total = amount + shipping_fee
11
12 print("Purchase amount:", amount)
13 print("Shipping fee:", shipping_fee)
14 print("Total payment:", total)
```

18.5 Example 5: Simple Login System

```
1 correct_username = "student"
2 correct_password = "python2026"
3
4 username = input("Username: ")
5 password = input("Password: ")
6
7 if username == correct_username and password == correct_password:
8     print("Login successful.")
9 else:
10    print("Incorrect username or password.")
```

A nested version can provide more detailed feedback:

```

1 if username == correct_username:
2     if password == correct_password:
3         print("Login successful.")
4     else:
5         print("Incorrect password.")
6 else:
7     print("Unknown username.")

```

18.6 Example 6: Triangle Validity

Three positive lengths can form a triangle when the sum of every two sides is greater than the third side:

[$a+b>c$, $a+c>b$, $b+c>a$.]

```

1 a = float(input("Enter side a: "))
2 b = float(input("Enter side b: "))
3 c = float(input("Enter side c: "))
4
5 if a <= 0 or b <= 0 or c <= 0:
6     print("Side lengths must be positive.")
7 elif a + b > c and a + c > b and b + c > a:
8     print("The three sides can form a triangle.")
9 else:
10    print("The three sides cannot form a triangle.")

```

18.7 Example 7: Triangle Type

After confirming that the sides form a triangle, classify it as equilateral, isosceles, or scalene.

```

1 a = float(input("Enter side a: "))
2 b = float(input("Enter side b: "))
3 c = float(input("Enter side c: "))
4
5 is_valid = (
6     a > 0 and
7     b > 0 and
8     c > 0 and
9     a + b > c and
10    a + c > b and
11    b + c > a
12 )
13
14 if not is_valid:
15     print("Invalid triangle")
16 elif a == b and b == c:
17     print("Equilateral triangle")
18 elif a == b or a == c or b == c:
19     print("Isosceles triangle")
20 else:
21     print("Scalene triangle")

```

The order is important. An equilateral triangle also satisfies the condition for an isosceles triangle, so the equilateral condition must be tested first.

18.8 Example 8: Electricity Bill

Suppose electricity is charged using the following simplified rule:

- first 100 units: 0.50 yuan per unit;
- next 100 units: 0.75 yuan per unit;

- units above 200: 1.00 yuan per unit.

```
1 units = float(input("Enter electricity usage: "))
2
3 if units < 0:
4     print("Usage cannot be negative.")
5 elif units <= 100:
6     bill = units * 0.50
7     print("Bill:", bill)
8 elif units <= 200:
9     bill = 100 * 0.50 + (units - 100) * 0.75
10    print("Bill:", bill)
11 else:
12    bill = 100 * 0.50 + 100 * 0.75 + (units - 200) * 1.00
13    print("Bill:", bill)
```

18.9 Example 9: Course Result with Validation

```
1 score = float(input("Enter a score from 0 to 100: "))
2
3 if score < 0 or score > 100:
4     print("Invalid score.")
5 elif score >= 90:
6     print("Grade A")
7 elif score >= 80:
8     print("Grade B")
9 elif score >= 70:
10    print("Grade C")
11 elif score >= 60:
12    print("Grade D")
13 else:
14    print("Grade F")
```

Validation should be performed before classification.

18.10 Example 10: Rock, Paper, Scissors

```
1 player_1 = input("Player 1: ").strip().lower()
2 player_2 = input("Player 2: ").strip().lower()
3
4 valid_choices = ("rock", "paper", "scissors")
5
6 if player_1 not in valid_choices or player_2 not in valid_choices:
7     print("Invalid choice.")
8 elif player_1 == player_2:
9     print("Draw")
10 elif (
11     (player_1 == "rock" and player_2 == "scissors")
12     or (player_1 == "scissors" and player_2 == "paper")
13     or (player_1 == "paper" and player_2 == "rock")
14 ):
15     print("Player 1 wins.")
16 else:
17     print("Player 2 wins.")
```

19 Common Errors and How to Correct Them

19.1 Error 1: Using = Instead of ==

Incorrect:

```
1 if x = 5:
2     print("x is 5")
```

Correct:

```
1 if x == 5:
2     print("x is 5")
```

19.2 Error 2: Missing the Colon

Incorrect:

```
1 if x > 0
2     print("Positive")
```

Correct:

```
1 if x > 0:
2     print("Positive")
```

19.3 Error 3: Incorrect Indentation

Incorrect:

```
1 if x > 0:
2 print("Positive")
```

Correct:

```
1 if x > 0:
2     print("Positive")
```

19.4 Error 4: Inconsistent Indentation

Avoid mixing tabs and spaces.

Incorrect or risky:

```
1 if x > 0:
2     print("Positive")
3 print("Number checked")
```

Correct:

```
1 if x > 0:
2     print("Positive")
3     print("Number checked")
```

19.5 Error 5: Comparing Different Data Types

Incorrect:

```
1 age = input("Enter your age: ")
2
3 if age >= 18:
4     print("Adult")
```

Correct:

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("Adult")
```

19.6 Error 6: Incorrect Use of or

Incorrect:

```
1 if answer == "yes" or "y":
2     print("Continue")
```

Correct:

```
1 if answer == "yes" or answer == "y":
2     print("Continue")
```

Better:

```
1 if answer in ("yes", "y"):
2     print("Continue")
```

19.7 Error 7: Overlapping Conditions in the Wrong Order

Incorrect:

```
1 if score >= 60:
2     print("Pass")
3 elif score >= 90:
4     print("Excellent")
```

A score of 95 satisfies the first condition, so the second branch is never reached.

Correct:

```
1 if score >= 90:
2     print("Excellent")
3 elif score >= 60:
4     print("Pass")
5 else:
6     print("Fail")
```

19.8 Error 8: Missing a Boundary

Incorrect for a valid score from 0 to 100:

```
1 if score > 0 and score < 100:
2     print("Valid")
```

This incorrectly rejects 0 and 100.

Correct:

```
1 if 0 <= score <= 100:
2     print("Valid")
```

19.9 Error 9: Unnecessary Comparison with True

The following is valid but unnecessarily long:

```
1 if is_member == True:
2     print("Member")
```

A clearer version is:

```
1 if is_member:
2     print("Member")
```

Similarly:

```
1 if not is_member:
2     print("Not a member")
```

is usually clearer than:

```
1 if is_member == False:
2     print("Not a member")
```

19.10 Error 10: Confusing Separate if Statements with elif

Consider:

```
1 score = 95
2
3 if score >= 60:
4     print("Pass")
5
6 if score >= 80:
7     print("Good")
8
9 if score >= 90:
10    print("Excellent")
```

This produces three messages. Use `elif` when only one category should be printed.

19.11 Error 11: Writing an Impossible Condition

The following condition can never be true:

```
1 if score < 0 and score > 100:
2     print("Invalid score")
```

A number cannot be below 0 and above 100 at the same time.

The correct condition is:

```
1 if score < 0 or score > 100:
2     print("Invalid score")
```

19.12 Error 12: Writing an Always-True Condition

Consider:

```
1 if score >= 0 or score <= 100:
2     print("Valid score")
```

This condition is true for almost every number.

For example:

- 150 satisfies `score >= 0`;

- -20 satisfies `score <= 100`.

The correct condition is:

```
1 if score >= 0 and score <= 100:
2     print("Valid score")
```

or:

```
1 if 0 <= score <= 100:
2     print("Valid score")
```

20 How to Design an if Statement

When solving a conditional problem, use the following process.

20.1 Step 1: Identify the Input

Ask:

- What information does the program need?
- What is the data type of each input?

Example:

The program needs a student's score, which is a number.

20.2 Step 2: Identify the Possible Outcomes

Example:

- Pass;
- Fail.

20.3 Step 3: Identify the Decision Rule

Example:

A score of at least 60 means Pass.

Translate it into Python:

```
1 score >= 60
```

20.4 Step 4: Choose the Correct Structure

- One optional action: use `if`.
- Two alternatives: use `if-else`.
- Several mutually exclusive categories: use `if-elif-else`.
- Several actions may all apply: use separate `if` statements.

20.5 Step 5: Check the Boundaries

For the passing score of 60, test:

[59, 60, 61.]

20.6 Step 6: Trace the Program

For each test value:

1. evaluate the condition;
2. determine whether it is true or false;
3. identify the executed branch;
4. determine the final output.